



Write the weak form at one quadrature point; get every operator, the MAP point and the posterior, on GPUs.

1 function differentiable

in JAX, the weak form at one quadrature point: **automatic differentiation** gives the residual, Jacobian, adjoint and every Hessian block. No derivative is coded by hand.

10^{-15} precision

relative agreement with **MFEM's own integrators** on the same quadrature rule: automatic differentiation is exact.

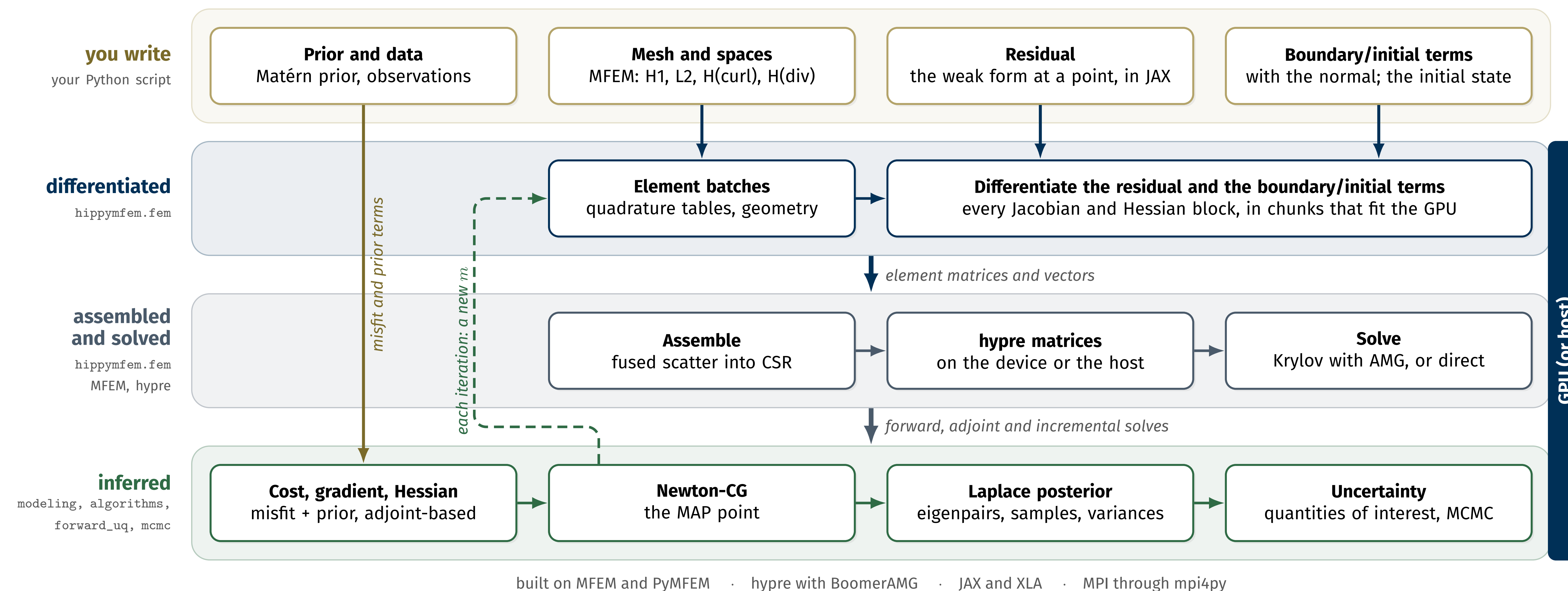
80× speedup

two Newton-CG steps with 287 M DoFs take **77 s on 16 RTX PRO 6000 GPUs**, against 103 min for hiPPYlibx on 32 CPU cores: 32 ranks and 9 CG iterations each.

1.09 billion DoFs

400³ P2 hexahedra, 514 M state + 514 M adjoint + 64 M parameter DoFs, on **24 RTX PRO 6000 GPUs**: a Newton-CG step in 197 s, JAX compilation included.

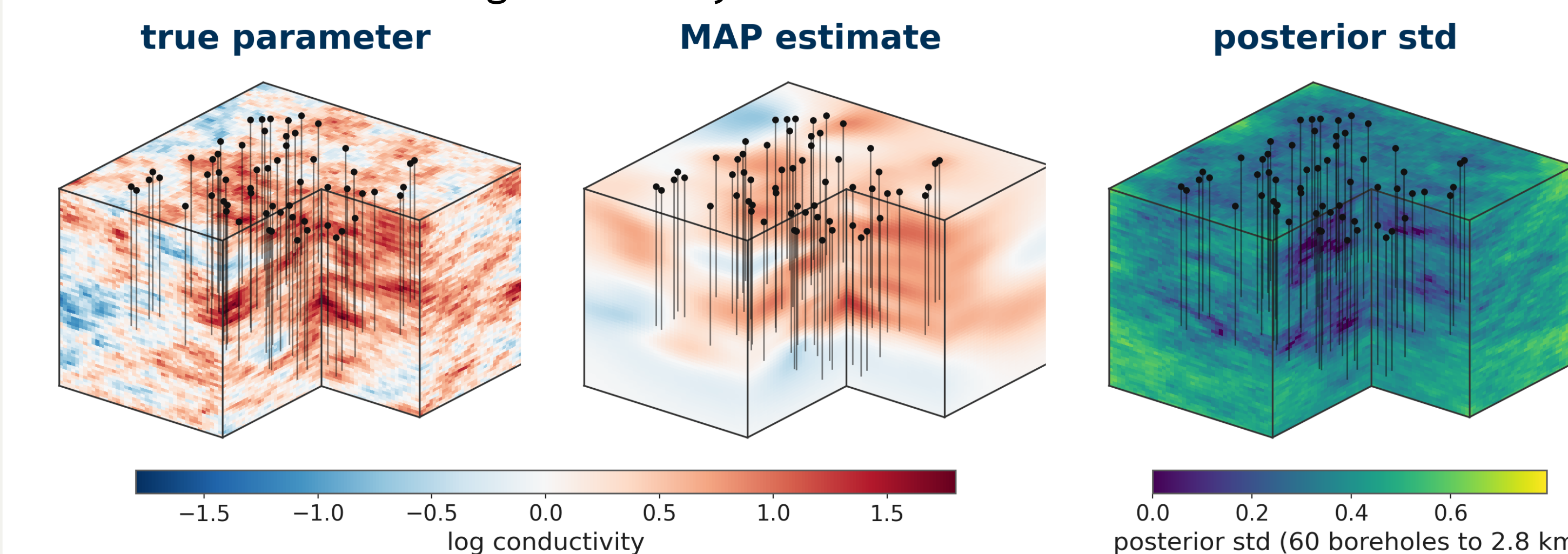
How it works: from the weak form to the posterior, on MFEM



The user writes the top row; every operator below it is generated or reused. No adjoint, Jacobian or Hessian block is ever written by hand, and the same code runs on the host or on GPUs.

Application: heat flow in an 8 × 8 × 4 km basin

Infer the rock's log conductivity from 60 boreholes logged to 2.8 km at 0.5 K. Conductivity is a **table per lithology**, read with a Gaussian kernel in seven lines of JAX: a nonlinear PDE no integrator library has.



on 4 L40S GPUs	64 ³ (4.6 M)	128 ³ (36 M)
MAP point	375 s	2761 s
low-rank Hessian, $k = 200$ and 50	157 s	253 s
one posterior sample	73 ms	242 ms
end to end, through the QoI	12.7 min	55.8 min

The body is found where the logs reach it (correlation 0.72), the truth within two posterior std of the MAP at **92.5 %** of the parameter DoFs and **93.1 %** at 128³, where MAP stops at the 30-iteration cap. The QoI carries 32 samples through the nonlinear map at 64³, 4 at 128³; the Monte Carlo variance is unbiased, 1.01 of exact at 16³.

Beyond the linearization. Posterior samples run **0.35 K hotter** than the MAP, 3.6 linearized std, since $E[1/k] > 1/E[k]$; a second-order Taylor term recovers $+0.39 \pm 0.02$ K at 38 % of the cost.

Write the physics once

```
mesh = mfem.ParMesh(comm, mfem.Mesh.MakeCartesian3D(
    64, 64, 64, mfem.Element.HEXAHEDRON))
Vu, Vm = hm.FunctionSpace.H1(mesh, 2), hm.FunctionSpace.H1(mesh, 1)

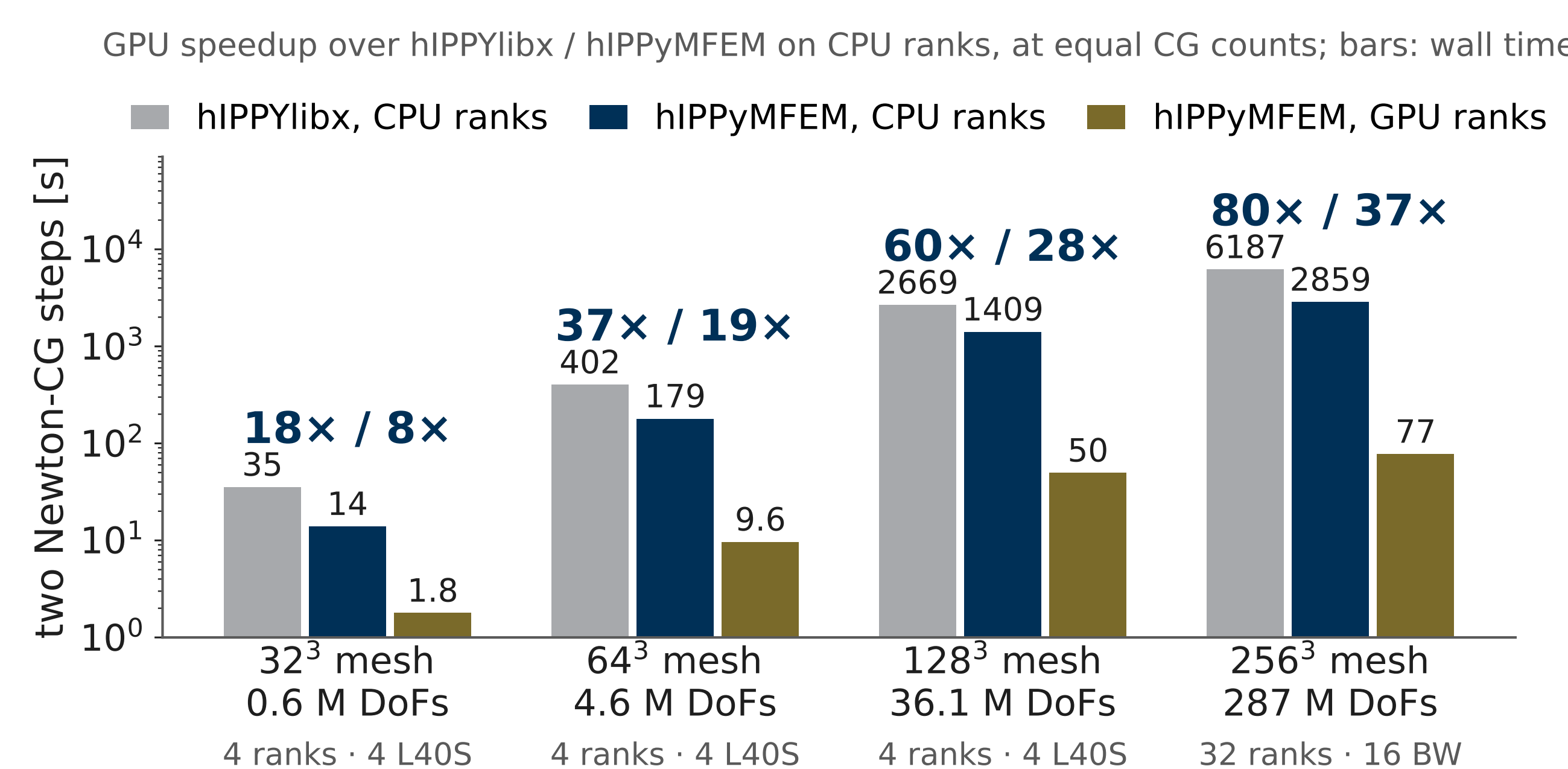
def pde_varf(u, m, p, x):
    # the weak form at a point
    return jnp.exp(m.val) * hm.inner(u.grad, p.grad) - f(x) * p.val

pde = hm.PDEVariationalProblem([Vu, Vm, Vu], pde_varf, bc, bc0)
prior = hm.BiLaplacianPrior(Vm, 0.1, 0.5)
model = hm.Model(pde, prior, misfit)
x_map = hm.ReducedSpaceNewtonCG(model).solve([None, m0, None])
```

- u, m, p carry the value and gradient (or curl, div) at the point. **Any twice-differentiable JAX code works**: smooth tabulated laws, neural-network closures, an inner Newton solve unrolled to a fixed number of steps.
- Every block is jacfwd of grad of that one function, vectorized over the elements, on **one quadrature rule**.

Validated, not asserted. Against hiPPYlibx on the same problem: 27 quantities agree, the Hessian spectrum to 3.7×10^{-13} . GPU against CPU: 4×10^{-16} . Finite-difference slopes: 1.000.

Faster: the whole Newton-CG step on GPUs vs CPUs



Above each group: **speedup of the GPU ranks over hiPPYlibx / hiPPyMFEM on as many CPU cores**. One node to 128³ (4 × L40S 46 GB, 2 × AMD EPYC 9334); at 256³, 32 of its 64 cores against 32 slices of 16 RTX PRO 6000 Blackwell (BW). Same PDE, prior and BoomerAMG settings; each CPU code is charged the GPU run's CG count, as the libraries stop CG at different counts.

Bigger: MFEM and hypre built for CUDA and ROCm

mesh	DoFs	GPUs	2 Newton-CG steps	fwd solve	CG step	per rank
64 ³	4.57 M	1 MI210	18.7 s	2.2 s	0.54 s	22.7 GiB
		1 L40S	27.9 s	3.3 s	0.62 s	21.7 GiB
128 ³	36.1 M	4 MI210	31.9 s	4.0 s	1.4 s	20.1 GiB
		4 L40S	49.5 s	6.1 s	2.2 s	19.1 GiB
256 ³	287 M	8 BW	140 s	15.5 s	4.4 s	33.5 GiB
		16 BW	77 s	8.5 s	2.6 s	19.7 GiB
400 ³	1.09 B	24 BW	197 s*	30.1 s	6.4 s	39.4 GiB

MI210: AMD Instinct, ROCm 7.2, hypre 3.2. BW: RTX PRO 6000 Blackwell, one rank per 48 GB MIG slice, two to a card (48 at 400³). A CG step is one reduced-Hessian action, two incremental solves. Read within a GPU count, not across: the BW rows declare dR/du symmetric and cap JAX lower. Memory is a rank's peak, measured in the process with its pool pinned across vendors on the 64³ and 128³ rows, polled from the driver on the BW rows, which reads higher. *One Newton-CG step, JAX compilation included.

Reference. P. Chen. *hiPPyMFEM: Differentiable programming for infinite-dimensional Bayesian inverse problems using MFEM on GPUs*. Preprint, 2026.

A weak form, not a form language.

Differentiating the pointwise residual gives exact, consistent operators on MFEM spaces for laws no grammar contains.

The whole inversion on the GPU.

With MFEM and hypre built for CUDA or ROCm, Newton-CG and the Laplace posterior run on GPUs: up to 80× over hiPPYlibx, rank for rank.

Next: Scaling to 4000³, 1.1 trillion DoFs.

17,000 GPUs of 128 GB, under half of Frontier or El Capitan, at 2 kB per unknown; expected: a few hours for likely the largest-scale nonlinear Bayesian inversion.